
DDL – Teil 1

BITLC | Tom Selig
7. August 2026

Aufgaben der DDL

- Die DDL ist eine der fünf Untergruppen von SQL.
- Datenstrukturen können erzeugt, geändert und gelöscht werden.
- Zu den Datenstrukturen gehören:
 - Datenbanken
 - Tabellen
 - Indizes
 - Views
- Es geht dabei lediglich um die Struktur einer Datenbank, nicht um die Daten selbst, die darin gespeichert werden sollen.

Anlegen einer Datenbank

- Auf einem Datenbank-Server können mehrere Datenbanken parallel existieren.
- Auf einem neu installierten SQL-Server gibt es immer vier bereits installierte Systemdatenbanken:
 - master => Interne Verwaltungsdaten des SQL Server
 - model => Vorlage zur Erstellung neuer Datenbanken
 - msdb => Steuerung regelmäßiger Aufträge
 - tempdb => Speicherung temporärer Daten
- Parallel dazu können weitere Datenbanken angelegt werden.

Anlegen einer Datenbank

- Eine neue Datenbank wird mit folgendem Befehl angelegt:

Syntax: `CREATE DATABASE <datenbankname>;`

Beispiel: `CREATE DATABASE testDB;`

- Dadurch wird eine Datenbank mit dem gewählten Namen erstellt.
- Der Name muss den Regeln für reguläre Bezeichner entsprechen:

Länge 1 bis 128 Zeichen

Erstes Zeichen muss ein Buchstabe oder `_` oder `@` oder `#` sein.

Weitere Zeichen können Buchstaben, Ziffern, sowie `_` `@` `#` `$` sein.

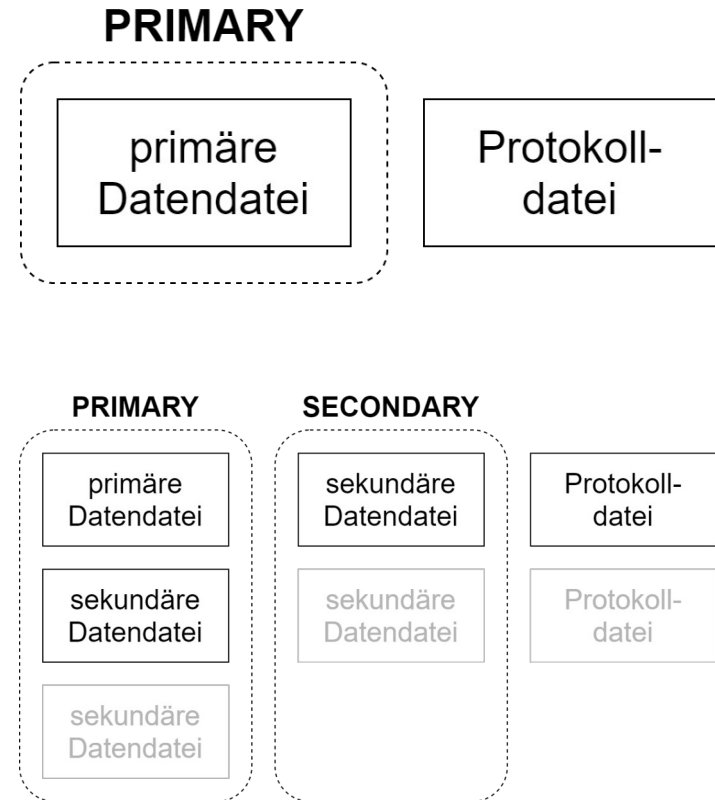
Auf Konventionen achten!

Anlegen einer Datenbank

- Der **CREATE DATABASE**-Befehl ohne weitere Angaben erzeugt die Datenbank unter Verwendung von Standardparametern.
- Eine Datenbank besteht beim SQL-Server immer aus mind. zwei physischen Dateien:

Primäre Datendatei in der Filegroup PRIMARY
(testDB.mdf)

Protokolldatei (testDB_log.ldf)



Ändern und Löschen einer DB

- Änderungen an einer bestehenden Datenbank können mit dem Befehl **ALTER DATABASE** durchgeführt werden.

Bisher noch nicht sinnvoll, da nur Standard-Werte genutzt.

- Das Löschen einer bestehenden Datenbank kann mit dem Befehl **DROP DATABASE** durchgeführt werden:

Syntax: **DROP DATABASE** [**IF EXISTS**] *<datenbankname>*;

Beispiel: **DROP DATABASE** [**IF EXISTS**] testDB;

- Löscht die Datenbank ohne Rückfrage. Eine Datenbank, die in Benutzung ist, kann nicht gelöscht werden. Zuerst müssen alle offenen Verbindungen gekappt werden.

Schemas

- Innerhalb einer Datenbank kann es mehrere Schemas geben. Sie werden primär zu zwei Zwecken verwendet:
 - logische Gruppierung von Objekten innerhalb der Datenbank
 - einfachere Verwaltung von Berechtigungen
- Die Hierarchie ist dabei folgende:
 - Server => Datenbank => Schema => Datenbank-Objekt
- Das Standard-Schema für neue Objekte (Tabellen etc.) ist **dbo**, was für Database Owner steht.
- Das **dbo**-Schema ist, neben einigen anderen, in jeder Datenbank immer vorhanden.

Schemas

- Ein neues Schema wird mit **CREATE SCHEMA** erstellt:

Syntax: **CREATE SCHEMA** *<name>*;

Beispiel: **CREATE SCHEMA** auswertungen;

- Löschen kann man ein Schema mit **DROP SCHEMA**. Das geht aber nur, wenn das Schema keine Objekte (Tabellen etc.) mehr enthält.

Syntax: **DROP SCHEMA** *<name>*;

Beispiel: **DROP SCHEMA** auswertungen;

- Die eingebauten Schemas wie **dbo**, **guest** oder **sys** können nicht gelöscht werden.

Anlegen einer Tabelle

- Innerhalb einer Datenbank können Tabellen mit dem **CREATE TABLE**-Befehl angelegt werden. Die minimale Syntax lautet:

Syntax: **CREATE TABLE** [*<schema>.*]*<tabellenname>* (
 <spaltenname> *<datentyp>*
 [, ...]
);

Beispiel: **CREATE TABLE** dbo.testTable (
 id **INT**,
 vorname **NVARCHAR**(20),
 nachname **NVARCHAR**(20),
 geburt **DATE**
);

- Zum Thema Datentypen gibt es eine separate Unterlage.

Anlegen einer Tabelle

- Weitere optionale Attribute für Spalten:

IDENTITY(n, m)

Sorgt für eine automatische Nummerierung der Spaltenwerte. Nur bei ganzzahligen Datentypen möglich. Startwert (n) und Schrittweite (m) können optional angegeben werden. Eine Tabelle kann nur über maximal eine mit **IDENTITY** markierte Spalte verfügen. Das **IDENTITY**-Attribut wird in der Regel bei Primärschlüsselspalten verwendet.

NULL | **NOT NULL**

Legt fest, ob die Spalte **NULL**-Einträge enthalten darf oder nicht. Wird die Angabe weggelassen, sind **NULL**s in der Spalte erlaubt. **NULL** ist das Kennzeichen für einen nicht vorhandenen Wert. **NULL** ist manchmal in der Handhabung schwierig, weil **NULL** in Vergleichen immer **UNKNOWN** ergibt.

Anlegen einer Tabelle (Constraints)

- Für einzelne oder mehrere Spalten können Einschränkungen (Constraints) formuliert werden.
- Die wichtigsten dabei sind zunächst die Schlüssel:
 - `PRIMARY KEY` (PK)
 - `FOREIGN KEY` (FK)
- Constraints können grundsätzlich auf Spalten- oder Tabellen-Ebene definiert werden. Wenn mehrere Spalten von der Constraint betroffen sind, muss auf Tabellen-Ebene definiert werden. Auf Spalten-Ebene können sich Constraints immer nur auf die eine Spalte beziehen, in der sie definiert werden.

Anlegen einer Tabelle (PK)

- Der Primärschlüssel kann auf Tabellen-Ebene definiert werden:

Syntax: [**CONSTRAINT** *<name>*] **PRIMARY KEY** (*<spalte>* [,...])

Beispiel: **CREATE TABLE** testTable (
 id **INT**,
 vorname **NVARCHAR**(20),
 nachname **NVARCHAR**(20),
 CONSTRAINT pk_testTable **PRIMARY KEY** (id)
);

- Die Angabe von **CONSTRAINT** *<name>* ist optional.
- Mehrere Schlüssel-Spalten können mit Komma getrennt in der Klammer angegeben werden.

Anlegen einer Tabelle (PK)

- Der Primärschlüssel kann auch direkt hinter der Spalten-Definition angegeben werden, wenn er sich nur auf diese Spalte bezieht:

Syntax: [**CONSTRAINT** <name>] **PRIMARY KEY**

Beispiel: **CREATE TABLE** testTable (
 id **INT CONSTRAINT** pk_testTable **PRIMARY KEY**,
 ...
);

- Auch hier ist die Angabe von **CONSTRAINT** <name> optional. Der SQL Server generiert dann selbst einen Namen.
- Bei mehreren Schlüssel-Spalten ist diese Variante nicht möglich, **PRIMARY KEY** darf nur hinter einer Spalten-Definition stehen.

Anlegen einer Tabelle (FK)

- Auf ähnliche Weise werden Fremdschlüssel in der Tabelle definiert:

Syntax: [**CONSTRAINT** <name>] **FOREIGN KEY** (<spalte> [,...])
 REFERENCES <Tabelle> (<spalte> [,...])

Beispiel: **CREATE TABLE** bestellung (
 kunde_id **INT**,
 ...,
 CONSTRAINT fk_bestellung_kunde
 FOREIGN KEY (kunde_id)
 REFERENCES kunde (id)
);

- Wichtig ist hier der Zusatz ab **REFERENCES**, denn damit wird die Verbindung zum referenzierten **PRIMARY KEY** hergestellt.

Anlegen einer Tabelle (FK)

- Auch der Fremdschlüssel kann direkt hinter der Spalten-Definition angegeben werden, wenn er sich nur auf diese Spalte bezieht:

Syntax: [**CONSTRAINT** *<name>*] **FOREIGN KEY**
 REFERENCES *<Tabelle>* (*<spalte>*)

Beispiel: **CREATE TABLE** bestellung (
 ... ,
 kunde_id **INT CONSTRAINT** fk_bestellung_kunde
 FOREIGN KEY REFERENCES kunde (id),
 ... ,
);

- Auch beim **FOREIGN KEY** ist die Angabe von **CONSTRAINT** *<name>* in beiden Varianten wieder optional.

Anlegen einer Tabelle (Beispiel)

```
CREATE TABLE author (  
  id INT NOT NULL,  
  firstname NVARCHAR(40) NOT NULL,  
  lastname NVARCHAR(40) NOT NULL,  
  CONSTRAINT pk_author  
    PRIMARY KEY (id)  
);
```

```
CREATE TABLE book (  
  id INT NOT NULL,  
  title NVARCHAR(100) NOT NULL,  
  isbn NCHAR(13) NOT NULL,  
  CONSTRAINT pk_book  
    PRIMARY KEY (id)  
);
```

```
CREATE TABLE authorbook (  
  author_id INT NOT NULL,  
  book_id INT NOT NULL,  
  CONSTRAINT pk_authorbook  
    PRIMARY KEY (author_id, book_id),  
  CONSTRAINT fk_authorbook_author  
    FOREIGN KEY (author_id) REFERENCES author (id),  
  CONSTRAINT fk_authorbook_book  
    FOREIGN KEY (book_id) REFERENCES book (id)  
);
```

Ändern einer Tabelle

- Tabellen können auch nachträglich verändert werden. Dazu dient der **ALTER TABLE**-Befehl.
- Der Befehl muss immer um eine zusätzliche Klausel ergänzt werden, die angibt was geändert werden soll:
 - ADD** => fügt eine Spalte zur Tabelle hinzu
 - ADD CONSTRAINT** => fügt eine Einschränkung zur Tabelle hinzu
 - ALTER COLUMN** => ändert eine vorhandene Spalte
 - DROP COLUMN** => löscht eine vorhandene Spalte
- Das Umbenennen einer Spalte oder der Tabelle geht mit dem **ALTER TABLE**-Befehl aber nicht!

Ändern einer Tabelle

- Das Hinzufügen von Spalten geschieht mit der **ADD**-Klausel:

Syntax: **ALTER TABLE** <table>
 ADD <spalte> <datentyp> [*<opt. Attribute>*];

Beispiel: **ALTER TABLE** kunde
 ADD gruppe_id **INT** NULL;

- Weitere Constraints wie z.B. Primär- oder Fremdschlüssel können mit der **ADD CONSTRAINT**-Klausel nachträglich hinzugefügt werden:

Beispiel: **ALTER TABLE** kunde
 ADD CONSTRAINT fk_kunde_gruppe
 FOREIGN KEY (gruppe_id)
 REFERENCES gruppe (id);

Ändern einer Tabelle

- Eine vorhandene Spalte kann mit der **ALTER COLUMN**-Klausel geändert werden:

Syntax: **ALTER TABLE** *<tabelle>*
 ALTER COLUMN *<spalte>* *<datentyp>* [*<opt. Attr.>*];

Beispiel: **ALTER TABLE** kunde
 ALTER COLUMN nachname **NVARCHAR**(50) **NOT NULL**;

- Das Löschen einer Spalte geschieht mit **DROP COLUMN**:

Syntax: **ALTER TABLE** *<tabelle>*
 DROP COLUMN *<spalte>*;

Beispiel: **ALTER TABLE** kunde
 DROP COLUMN geburt;

Ändern einer Tabelle

- Einige Operationen lassen sich im SQL-Server nicht über die Standard-Befehle umsetzen, z.B. das Umbenennen einer Spalte.
- Für solche Fälle gibt es im SQL-Server sog. Stored Procedures (SPs), die mit dem Befehl EXEC aufgerufen werden:

Syntax: EXEC sp_rename '<namealt>', '<nameneu>', 'COLUMN';

Beispiel: EXEC sp_rename 'kunde.geburt', 'gebdat', 'COLUMN';

- Mit **sp_rename** kann man auch den Namen von Datenbanken ändern, das letzte Argument lautet dann **'DATABASE'**.
- Um den Namen einer Tabelle zu ändern, muss das letzte Argument komplett weggelassen werden.

Löschen einer Tabelle

- Das Löschen von Tabellen geschieht mit dem **DROP TABLE**-Befehl:
Syntax: **DROP TABLE** [**IF EXISTS**] *<tabelle>* [, ...];
Beispiel: **DROP TABLE** kunde;
- Dabei können auch mehrere Tabellen gleichzeitig gelöscht werden.
- Achtung: Es erfolgt keine Rückfrage und die Aktion kann nicht rückgängig gemacht werden!
- **DROP TABLE** funktioniert nicht, wenn noch ein **FOREIGN KEY** auf die Tabelle verweist. Dieser muss vorher entfernt werden.
- **IF EXISTS** verhindert Fehler, wenn die Tabelle nicht existiert.